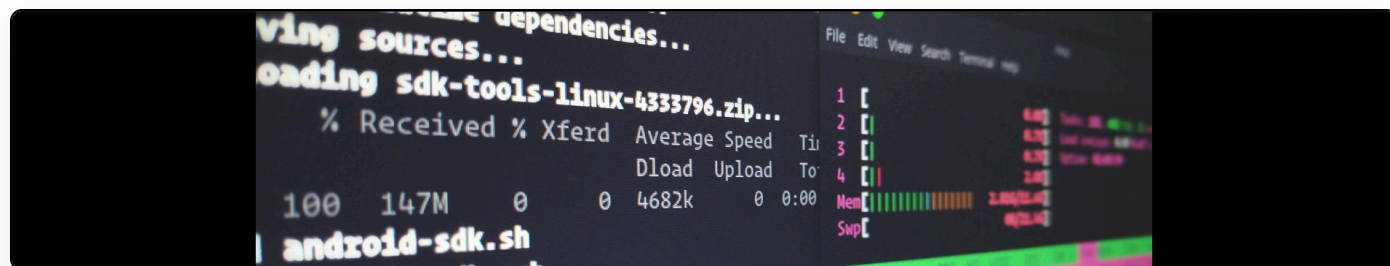


Modularizing the meTypeset typesetter for the user interface phase

Cite as: Martin Paul Eve, "Modularizing the meTypeset typesetter for the user interface phase", *eve.gd: Martin Paul Eve*, February 24, 2014, <https://doi.org/10.59348/wp7-g2j17>.



Modularizing the Project

Today brings with it some notable changes to my scholarly article XML (NLM/JATS) typesetter ([meTypeset](#)). First off, the project is now nicely [handling user supplied captions](#) to figures, so long as they are in the format “Figure 1: a figure caption”.

The second, more important change, that I've implemented, however, is to begin to modularize the project and to add individual command line hooks for different functions. So it's now possible to run specific functions. This will allow us to begin to build a user interface on top of the software.

The envisaged process at the moment is:

1. web service calls meTypeset on document
2. user is presented with WYSIWYG editor/step-by-step walkthrough to fix aspects

The way in which the step

The reference linker

A good example of this is the new options available via the reference linker. The reference linker is the portion of meTypeset that scans through documents looking for parenthetical references (Eve 2014 etc.) and then tries to link such items to valid ref-list/ref entries.

We have a moderately good success rate on such aspects. If it's done properly, with semicolons between bracketed entries and there is a single ref-list/ref entry that matches all the space separated and special-character normalized components of the reference, then we can link it no problem.

If, however, there are entries we can't parse, we need a user interface to handle this. While this will be web-based eventually, I today added an `-interactive` flag to the typesetter that will allow a command line input aspect, like this:

 [meTypeset interactive](#)

Figure 1: meTypeset running in interactive mode

This interface, which is based on the excellent [beets music tagger](#), will be of great personal use to me when I typeset articles for [Orbit](#) (and for my hoped-for move to [lens viewer](#) at some point). Basically, here the user can select a specific option (Skip, Delete (remove the stub xref element), Enter search (>search the reference list for the correct entry), enter Link id (give an absolute element ID attribute to function as the rid attribute), or Abort the entire process).

This is just a start, but if we can develop a slick UI for the web on top of this, we will be solving a substantial problem in the currently commercial toolchain.

Cite this article

Please include the DOI in your citation: <http://dx.doi.org/10.6084/m9.figshare.942425>

You can [view this post's XML with lens](#).